

The firm's system of record, **born headless.**

A records and matter management platform for the practice of law.

Lawyers and paralegals serve themselves; policy decides which actions need a second set of eyes; every consequential act lands in an anchored audit trail that carries no content. No UI, by design: any interface, bot, or integration builds on the same API.

- Headless API

- Policy-routed oversight

- Anchored audit trail

- Jurisdiction-aware retention

01 What this is

This is a headless platform that manages the firm's records across every series: contracts, civil litigation, client files for people and for organizations, and whatever the file plan defines next. It rests on a strict separation of five ideas that most systems blur. A **document** is content: bytes, versions, renditions. A **record** is a declaration: a specific document version fixed as evidence. A **filing** is the act of placing a record into a file, and it carries the series classification, so the same record can be litigation material in one file and contract material in another. A **file** is the dossier lawyers open and close. A **matter** is the engagement, with a client, a responsible attorney, and a conflict check.

Because the series lives on the filing, **one record can sit in several files under different rules without being copied**. Each filing projects its own disposition date from its own series trigger, and the record's obligation is the latest projection across all of them. Access is the union of the files it sits in, intersected with the record's own classification floor, so cross-filing can never widen who can reach something. A screen on any filing suppresses the record for that person everywhere.

Every filing **answers for itself**. Who filed it, when, by what mechanism, at what confidence, against which schema version, under which series. You can answer why a record is in a file from that filing alone, without reconstructing the record's whole history. Filings are superseded or withdrawn, never deleted, and withdrawing one does not retract an obligation it already put in force.

Declaration is precise about what it does. It **freezes the bytes and the fact of the declaration**, and nothing else. Metadata is corrected additively, classification changes per filing, privilege is effective-dated because privilege can be waived or lost, renditions are added over time, and retention position recomputes continuously. A later document version becomes a new record with a supersedes link, never an edit to the old one.

It is **self-service by default, governed by policy**, under one lifecycle contract. Every stateful resource declares its states, its transitions, and its guards. Every change goes through the same action envelope, and the platform answers what a given caller may legally do next, so no client hardcodes the state machine.

And it draws a firm line about truth. A durable **system of record holds current state with full temporal versioning**. The event log is the anchored, append-only account of what happened around it, carrying identifiers, hashes, decisions, and actors, but never content. That is what lets the firm honor a destruction certificate without breaking its own audit trail.

02 What it does

The capability in plain terms, described by what people get rather than how it is built.



Self-service file lifecycle

Lawyers and paralegals open, close, merge, transfer, and reopen files through one action endpoint, without waiting on a records queue for routine work.



Oversight where it counts

Sensitive actions route to a reviewer automatically: closing a file under hold, merging across matters, reclassifying a series, releasing a hold. The requester can never be the approver.



One record, different rules

A record can be litigation material in one file and contract material in another, because the classification rides on the filing. No copies, no reconciling two series into one answer.



Cross-filing cannot widen access

Filing a record somewhere less protected does not expose it. Reach is the union of its files, capped by the record's own floor, and any screen suppresses it everywhere.



Drafts protected from day one

Working documents carry the same access control, walls, and privilege designation as declared records. Nothing waits on a filing decision to be confidential.



Every filing answers for itself

Who filed it, when, how, at what confidence, under which series. One filing's record explains why the document is there, without reading anything else.



Declaration freezes the bytes

Fixing a record pins one document version and the fact of the declaration. Corrections, reclassification, and later privilege determinations remain possible, with history.



One party graph

People and organizations resolved once, across every matter, with each claim traced to its source. Conflicts checks and ethical walls run against real identities.



Auto-filing with a dial

Classification lands as a scored claim. High confidence files itself; the uncertain tail queues for review. No classifier decision can move a record onto a shorter clock.



Retention that fits the law

Trigger-based rules, not flat durations. Each filing projects its own disposition date, and an unfired trigger holds the record rather than quietly expiring.



Obligations only ratchet up

Cross-filing, reclassification, and merging can lengthen how long something must be kept. None can shorten it, and withdrawing a filing leaves its obligation behind.



Holds that keep up

Legal holds are live queries: newly filed records that match join the hold automatically. Nothing under hold can be destroyed, and release is a reviewed act.



Search that respects walls

Every query is trimmed against ethical screens and matter access at the moment it runs. A screened attorney's search simply never sees what it must not.



Destruction that really deletes

Content goes; a tombstone stays, with the dates, the authority, and the certificate. Nothing about the audit trail depends on keeping what the firm agreed to destroy.



An audit others can verify

The event log's digests are anchored to storage the platform cannot rewrite, and the verifier runs outside the platform team. Self-attested audit trails prove nothing.

03 How much oversight does an action get?

Oversight is policy-routed, not caller-chosen. A paralegal and a records manager hit the same endpoint; the platform decides whether a second set of eyes is required. What differs by action class is how much review stands between the request and the result.

Auto

EXECUTES NOW

POLICY ALLOWS • SYNCHRONOUS • 200

The bulk of daily work: opening a file in a routine series, filing a record, adding a party. The action **executes immediately**, the event publishes, and nobody waits on anybody.

Review

ONE APPROVER

Actions with consequences: closing a file, transferring between portfolios, low-confidence auto-filing. The request returns a workflow, lands on a **reviewer's worklist**, and executes on approval. Requester and approver can never be the same person.

Govern

ROLE-GATED

The acts the firm answers for: merging files across matters, reclassifying a filing's series, withdrawing a filing, releasing a legal hold, certifying destruction, resolving a disposition method conflict. Reserved to **records management or general counsel roles**, with the decision, the deciding identity, and the certificate all preserved permanently.

NOTE All actions carry idempotency keys, and pending workflows take a lease on the resources they affect, so two people racing to merge the same file into different targets resolve cleanly rather than corrupting state.

Policy decides, not the caller

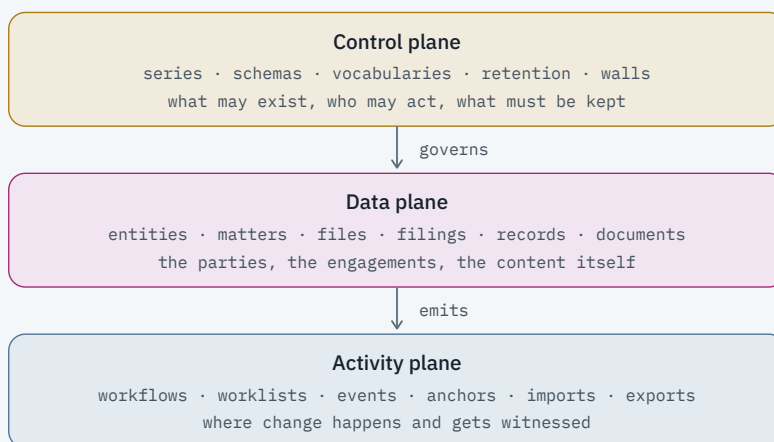
Lifecycle transitions are requested, not performed: **POST /{resource}/{id}/actions** with a type of open, close, merge, transfer, reclassify, withdraw, or dispose. The same envelope covers every stateful resource, not just files. Guards evaluate in a fixed order, existence then authorization then state legality then invariants, before the policy engine decides routing. If no oversight is required, it executes synchronously. If review is required, the platform opens a workflow and routes it. Adding a new controlled action later, or tightening review on an existing one, is configuration owned by the records team, not an API redesign owned by engineering. Self-service stays the default; oversight is an overlay.

04 How it works

Six views of the system, each with a plain-language read. The technical reference carries the full detail.

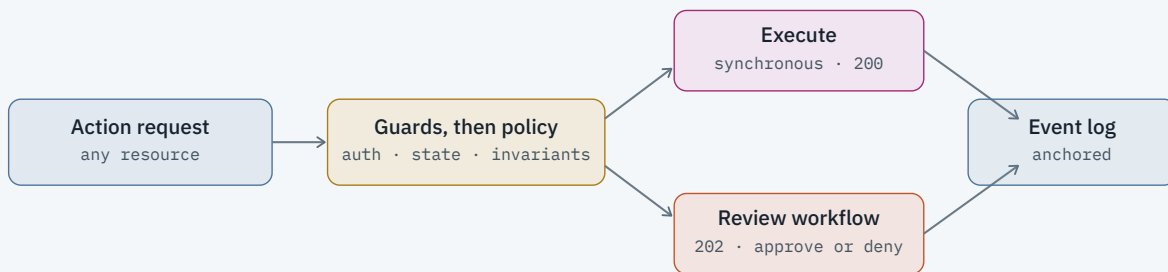
A Three planes

The control plane holds configuration owned by records managers: the file plan, metadata schemas, vocabularies, retention rules, and walls. It governs the data plane, where the substance lives: the party graph, matters, files, filings, records, and documents. Every change in the data plane emits into the activity plane, where workflows run, worklists queue, and the anchored audit log accumulates. The control plane is what makes the platform configurable by the records team instead of engineers; the activity plane is what makes self-service safe.



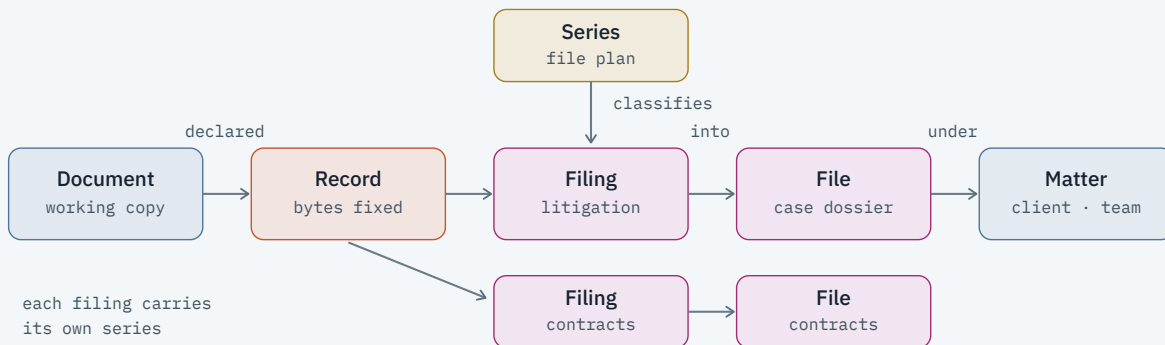
B One lifecycle contract

Every stateful resource in the platform, not just files, declares its states, its legal transitions, and its guards. A request hits the action endpoint with an idempotency key. Guards evaluate in a fixed order: existence, then authorization, then state legality, then invariants, then policy routing. That order is part of the contract, because it decides what a caller learns from a refusal: someone who should not know a resource exists cannot infer its state from an error message. Routine actions execute and return the new state; controlled actions return a workflow. A caller can also ask what it may legally do next, so no client hardcodes the state machine.



C The series rides on the filing

The ontology the whole platform stands on, and the detail that makes the rest of it work. A document is mutable content. Declaring it produces a record: one document version, fixed. A filing places that record into a file and carries the series classification, which is why the same record can be litigation material in one file and contract material in another with no copy and no conflict to reconcile. Files open under matters. Each filing projects its own disposition date from its own series trigger, and each one is independently auditable.



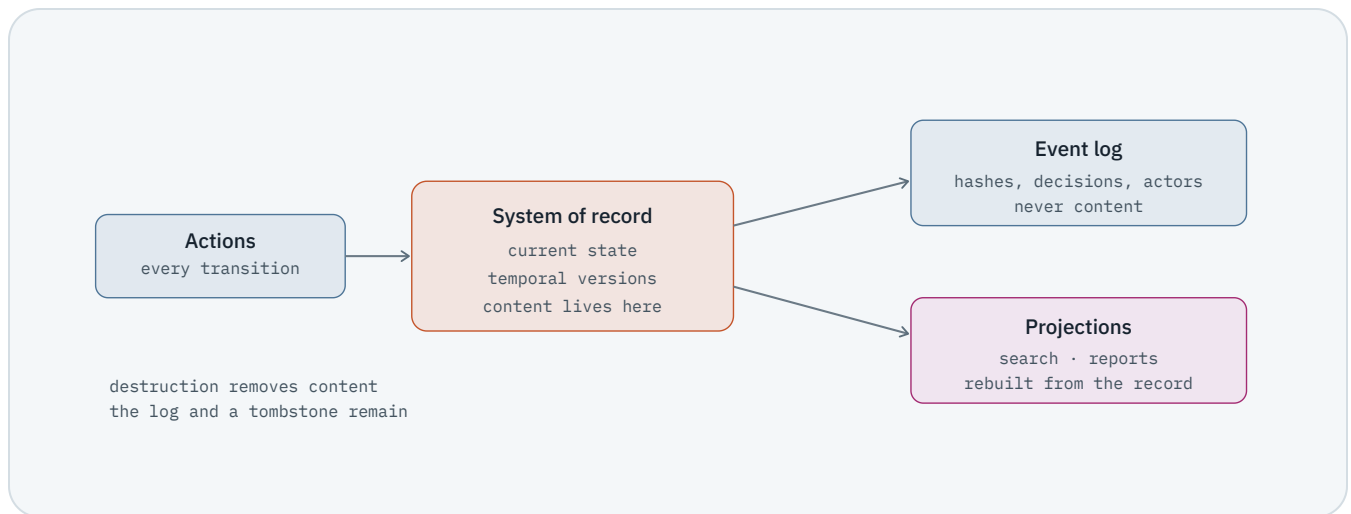
D Retention only ratchets up

A platform invariant, not a rule that lives in one place. Each filing projects a disposition date from its own series trigger, and the record's obligation is the latest projection across all of them. Durations are not compared directly, because triggers differ: an event-based trigger that has not fired yet holds the record rather than contributing a number. Cross-filing never accelerates destruction, reclassification applies only to what is filed afterward, and withdrawing a filing leaves its obligation in force. Disposition then requires that every filing has cleared and no hold applies, re-checked inside the transaction that destroys.



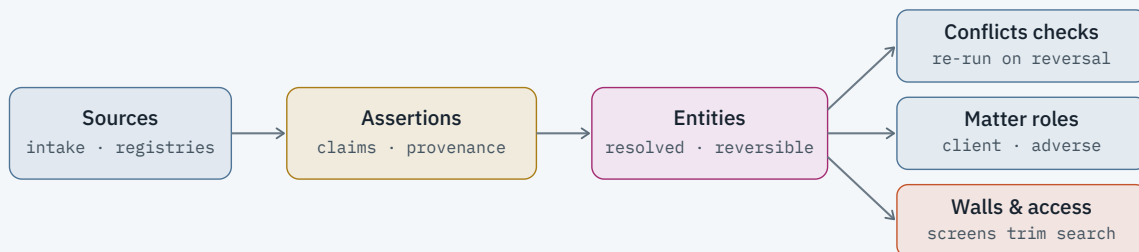
E Where truth lives

A deliberate decision, not a default. A durable system of record holds current state with full temporal versioning, and it is where content lives. The event log is the anchored, append-only account of what happened around it. Events carry identifiers, versions, content hashes, decisions, actors, and reason codes, and never content itself. That constraint is what makes lawful destruction possible: removing the bytes invalidates no event, because no event held them, and the hash left behind proves what was destroyed without retaining it. Projections rebuild from the system of record rather than by replaying the log, so a destroyed record cannot reappear in search. The party graph is the one deliberate exception, where accumulating claims really are the truth and resolved identities are the projection over them.



F One party graph, many consumers

Identity is built from claims, not overwrites. Every source, intake forms, corporate registries, migrated legacy systems, the auto-classifier, contributes assertions with provenance and confidence. Entity resolution fuses assertions into resolved identities, and because the claims are preserved, a merge can be reversed in the graph. Reversing one is a correction workflow, not a button: the platform inventories every matter, wall, filing, and conflicts clearance that referenced the merged identity, marks affected clearances provisionally invalid, and holds the separation open until a human has resolved each reference.



The audit log is tamper-evident, not tamper-proof

A hash chain the platform writes and the platform validates protects against nothing an attacker with database access cares about, because they can rewrite the chain. So the chain is only the inner layer. At a fixed interval a separate principal computes a Merkle root over the events since the last anchor and writes it to object-lock storage the application runtime holds no credentials for. A verifier that reads the log and the anchor store runs **outside the platform team**, ideally by internal audit, because a log only the platform can validate proves nothing about the platform. Certificates of destruction additionally carry an external RFC 3161 timestamp, which turns an internal claim into something with standing. Between anchors there is a window, and the anchoring interval is a stated number, not a promise of continuity.

05 The API surface

Resource families grouped by plane. Everything below is REST plus a webhook-subscribable event stream; there is no other way in, which is the point.

CONTROL PLANE

FILE PLAN

/series

The record series taxonomy. A series is applied by a filing rather than held on the record, so the same record can be classified differently in different files.

METADATA PROFILES

/schemas

Versioned required and optional fields per series. Records validate at declaration time against the schema version in force, and reclassification never invalidates them retroactively.

REFERENCE DATA

/vocabularies · /terms

Controlled terms with effective dates, deprecation, and superseded-by chains. Crosswalks map terms to external systems. Never hard-deleted once referenced.

RETENTION RULES

/retention-schedules

Trigger, duration, and disposition method, jurisdiction-aware. Each filing projects its own date; an unfired trigger holds rather than expires; conflicting methods escalate instead of resolving silently.

ETHICAL SCREENS

/walls · /policies

Queryable, auditable screens and the routing rules that decide which actions need review. Walls override team membership everywhere.

NUMBERING

/numbering-schemes

Client-matter-file number formats as pluggable configuration, with the scheme id stored alongside every generated number for acquisition-era coexistence.

DATA PLANE

PARTY GRAPH

/entities · /identifiers

People, organizations, courts, and agencies as resolved identities. Losing ids from merges persist forever as redirects.

CLAIMS

/assertions · /sources

Every fact about an entity as a sourced, scored claim. This subsystem is genuinely claims-first: resolved entities are a projection over accumulating assertions.

CONNECTIONS

/relationships

Typed links across the graph: employment, ownership, representation, place-in-hierarchy. The substrate conflicts checks traverse.

ENGAGEMENTS

/matters

The engagement with client, conflict status, and role-bearing party links. Matter teams at `/matters/{id}/team` carry delegated authority.

ENGAGEMENT FACTS

/matters/{id}/events

What happened on the engagement, including court filings, hearings, and service dates. Distinct from internal filings, which place records into files.

CONTENT

/documents

Working, declared, or transient. Mutable content with versions and renditions, content-addressed and fixity-checked. Access, walls, and privilege attach here, not at declaration.

DECLARATIONS

/records

Fixes one document version by hash, plus the fact of the declaration. Metadata stays correctable, privilege stays effective-dated, renditions stay additive, and a later version becomes a successor record.

PLACEMENT

/filings

Carries the series, position, filer, mechanism, confidence, and schema version. Exactly one filing per record is primary. Superseded or withdrawn, never deleted, and withdrawal leaves a residual obligation.

PLACEMENT AUDIT

/filings/{id}/events

Each filing's own stream, so why a record sits in a file is answerable from that filing alone, without reconstructing the record's whole history.

DOSSIERS

/files

The aggregation containers lawyers open and close. Belong to a matter. GET returns current state only, never a segment the caller has to reason about.

HISTORY

/files/{id}/lifecycle

The ordered open and close segments with their reasons and retention behavior. An audit structure, deliberately not a query interface for reporting.

PRESERVATION

/holds

Legal holds as continuously evaluated scope queries with tracked notifications, reviewed release, and conflict reporting across overlapping holds.

DESTRUCTION

/dispositions

Runs only when every filing has cleared and no hold applies, re-checked inside the destroying transaction. Content goes; a tombstone and a timestamped certificate persist.

ACTIVITY PLANE

TRANSITIONS

/resource/{id}/actions

One envelope for every stateful resource. GET returns the transitions this caller may legally take now and whether each executes or routes to review, so no client hardcodes the state machine.

OVERSIGHT

/workflows

Pending decisions with separation of duties enforced in the engine and leases held on affected resources until resolution.

REPAIR

/corrections

Long-running workflows for reversing a merge or a mistaken reclassification: impact inventory first, every downstream reference resolved by a human, commit only when none remain.

QUEUES

/worklists

Pending approvals, disposition reviews, classification queues, expiring holds, unresolved corrections. What any UI or bot renders as an inbox.

AUDIT

/events

The append-only account of what happened around the system of record. Identifiers, versions, hashes, decisions, actors, reason codes. Never content, which is what makes lawful destruction possible.

ATTESTATION

/anchors

Periodic Merkle roots over the event log, written by a separate principal to storage the application cannot rewrite, with an independently runnable verifier.

INGEST

/imports

Migration and bulk ingest with staging, schema validation, quarantine, and provenance stamping. Legacy metadata arrives as sourced assertions.

PRODUCTION

/exports

Discovery load files, client transfer packages, regulator productions. Bates numbering, privilege enforcement, and a log of what left and under what authority.

FINDABILITY

/search · /reports

Search is security-trimmed per result at query time. Both rebuild from the system of record, so destroyed content cannot reappear in an index.

06 Roadmap

Build order grouped by dependency rather than appeal. The ontology and file plan come first because every other capability classifies against them; the automation comes after the workflow machinery it rides on.

NOW · DELIVERED DESIGN

In design

Platform architecture

The ontology, the truth model, the API surface, and the oversight model are settled. The five-concept ontology with the series on the filing; declaration that freezes bytes and nothing else; a system of record with a content-free audit log around it; the ratchet invariant; and one lifecycle contract across every resource are the load-bearing decisions everything below assumes.

Series on the filing

Declaration semantics

System of record + audit log

Retention ratchet invariant

One lifecycle contract

PHASE 1

Foundation

Ontology & file plan

The substrate everything else rides on: series, versioned schemas, and vocabularies in the control plane; matters, files, filings, records, and documents in the data plane; the system of record with temporal versioning, and the content-free anchored event log around it. Document states, per-filing series, primary and cross-filings, each filing's own audit stream, and current-state projections all land here, because retrofitting either the truth model or the audit spine is not a real option.

Series & schemas

Working, declared, transient

Filings with their own series

Per-filing audit streams

Temporal system of record

Anchored event log

Actions, oversight & corrections

The lifecycle contract made real across every resource: state and transition tables, the fixed guard order, centrally checked invariants, discoverable transitions, review workflows with separation of duties, resource leases, and worklists. Merge, reclassification, filing withdrawal, and the correction workflows that reverse them ship together here, because a firm that can merge but cannot repair accumulates unfixable mistakes in the gap.

Transition tables

Fixed guard order

Central invariants

Discoverable actions

Merge & reclassify

Correction workflows

Party graph & conflicts

Claims-based entity resolution: entities, identifiers, assertions with source provenance, relationships, and entity merge. This is the one subsystem where accumulating claims genuinely are the truth and resolved identities are the projection. Conflicts checks run at matter open, and ethical walls become queryable resources composed with matter teams. Entity merge cannot ship ahead of the correction workflow it depends on.

Claims as truth

Entity resolution

Conflicts at open

Walls & matter teams

Merge reversal

PHASE 4

Needs P1 + P2

Retention, holds & disposition

The governance payload: the retention rule expression model with jurisdiction inputs, per-filing date projection with the latest winning, escalation when disposition methods conflict, continuously evaluated holds, and the full disposition chain through certified destruction with an external timestamp and a surviving tombstone. This is the phase that turns the platform from a filing system into a defensible records program.

Retention expressions

Per-filing projection

Method conflict escalation

Live hold scoping

Tombstones & certificates

PHASE 5

Needs P2 + P4

Auto-filing

Classification as scored assertions on the filing: above a per-series threshold the record files itself and the event publishes; below it, the same workflow engine queues it for review. Thresholds reflect the asymmetry of misfiling, and the ratchet invariant means no classifier decision, at any confidence, can place a record onto a shorter clock. Model and threshold versions are control-plane resources, so a decision can be read back against the version that made it.

Scored classification

Per-series thresholds

Never shortens retention

Versioned models

PHASE 6

Needs P1 · scales with P4

Import & export

Migration as a product feature: staged batch imports with validation, quarantine, and provenance stamping, so decades of legacy records arrive as sourced assertions rather than laundered facts. On the way out, discovery productions, client file transfers, and regulator packages with Bates numbering and privilege enforcement. A migration spike against a real corpus sample belongs in Phase 1, well ahead of this, to test the ontology before it hardens.

Staged migration

Provenance stamping

Discovery productions

Client transfers

07 Decided, and still open

The commitments the design makes, and the questions the next sessions need to close. This is a design direction that has survived two adversarial reviews, not a specification: treat it as largely settled in shape, pending validation against a real corpus and a named first consumer for the API.

LOCKED IN

- ✓ Document, record, filing, file, and matter are five distinct resources, and the series classification lives on the filing rather than the record
- ✓ One record, many filings: exactly one is primary, each projects its own disposition date, and the latest projection governs
- ✓ Access is the union of a record's files intersected with its own classification floor, so cross-filing can never widen reach
- ✓ Every filing is independently auditable and answers on its own why a record sits in a file
- ✓ Filings are superseded or withdrawn, never deleted, and withdrawal leaves a residual obligation in force
- ✓ Declaration freezes the bytes of one document version and the fact of the declaration; metadata, classification, and privilege stay versioned
- ✓ A later document version becomes a successor record with a supersedes link, never an edit to the original
- ✓ Retention obligations only ratchet up, and an unfired trigger holds a record rather than quietly expiring
- ✓ Conflicting disposition methods escalate to a records manager; only permanent retention wins automatically

- ✓ One lifecycle contract across every stateful resource, with a fixed guard order and transitions the platform will enumerate on request
- ✓ A temporal system of record holds truth; the event log is the anchored account around it and carries no content
- ✓ Destruction removes content and leaves a tombstone; projections rebuild from the record, not by replaying the log
- ✓ The party graph is the deliberate exception, where accumulating claims are the truth and entities are the projection

OPEN QUESTIONS

- The retention rule expression language: how rich, how jurisdictional, and who authors it
- Whether the party graph is directly readable or conflicts runs as a privileged service returning a decision rather than the graph
- What destruction means against backups, replicas, and delivered exports, and whether crypto-shredding is required for the certificates to be true
- How long tombstones live, and whether metadata retention is set per series or firm-wide
- Precedence beyond permanent retention when two series prescribe different disposition methods, if any is safe to automate
- The anchoring interval, and what exposure the window between anchors represents
- Aging policy for undeclared documents on closed matters: bulk declaration under a catch-all series, or purge
- How matter teams, firm roles, and walls compose into one authorization decision, and what sets a record's classification floor

- Who designates a filing as primary, and what happens when the primary file closes while a cross-filing stays open
- The event schema catalog and versioning policy for the stream integrators build on
- Storage tiering and WORM guarantees for declared records in regulated-client work

Platform overview · headless legal records & matter management · REST + event stream · revision C · post-review · design reference, not a commitment